

SOFT COMPUTING · UNIT IV

Artificial Neural Networks

Learning guide for Group 12 — every slide in plain English, with analogies

How to use this guide. Read it once straight through (about 25 minutes), then read only the slides you will present. Each slide has four parts: what it says in simple words, an everyday **analogy** you can reuse while speaking, and a “**say it like this**” sentence you can use as your opening line. You do not need to memorise formulas — you need to understand the *idea* and be able to point at the slide and walk through it.

The whole unit in one paragraph

Your brain is made of cells called neurons. Scientists copied the idea into a very small maths machine: take some numbers in, multiply each by an “importance” number (a **weight**), add them up, and decide an answer. Connect many such machines in layers and you get a **neural network**. A network starts out knowing nothing, because its weights are random. **Learning** just means nudging the weights, again and again, until the answers become right. Different *learning rules* say how to nudge. **Backpropagation** is the method that makes this work for big, multi-layer networks. A different type of network, the **Hopfield network**, works as a memory that can repair damaged patterns. Finally, these ideas are used in recognition, forecasting and image work.

Map of the 18 slides

Part	Slides	What you will be able to explain
1 · Foundations	1-5	Biological neuron, artificial neuron, activation, layers
2 · Learning	6-13	Learning rules, three paradigms, perceptron, delta rule, MLP, backpropagation
3 · Memory & uses	14-18	Hopfield network, associative memory, three applications, takeaways

Suggested split for 4 speakers (about 4-5 slides each, balanced by difficulty)

Speaker	Slides	Topic
Member 1	1-5	Intro, biology, artificial neuron, architecture
Member 2	6-9	Learning rules, paradigms, perceptron, worked example
Member 3	10-13	Delta rule, MLP, backpropagation (the hardest part — start with this person’s revision)
Member 4	14-18	Hopfield, associative memory, applications, conclusion

Four words you will hear again and again

- **Weight (w)** — how important an input is. Learning = changing weights.
- **Bias (b)** — a built-in nudge that makes a neuron more or less eager to fire.
- **Activation function (f)** — the final “decision” step that turns the total into an output.
- **Error** — how far the network’s answer is from the right answer. Learning tries to shrink it.

Part 1 · Foundations

Slides 1–5: from a real neuron to an artificial one

SLIDE 1 Title — Artificial Neural Networks

This is the opening slide for Unit IV. The subtitle says the journey: from **biological neurons** (what is in your head) to **learning machines** (what we build with maths). The badge “Group 12” opens the team list.

ANALOGY Think of it as the trailer of a film: you are about to see how nature’s idea was turned into a machine that can learn.

SAY IT LIKE THIS “Good morning, sir. Today, Group 12 will explain how a copy of the brain’s basic cell, the neuron, can be built with simple maths, how it learns, and where it is used.”

SLIDE 2 It Starts With a Neuron

Your brain has about 86 billion neurons. Each one is a tiny cell that listens to many other cells and passes a signal on. Parts on the slide:

- **Dendrites** — the “ears”: they receive signals from other neurons.
- **Soma** (cell body) — the “decision maker”: it combines all incoming signals.
- **Axon** — the “wire” that carries the output signal away. The **myelin sheath** is its insulation, which makes the signal travel faster.
- **Synaptic terminals / synapses** — the tiny gaps where one neuron hands the signal to the next. Signals are electrical inside the neuron and chemical across the gap.

ANALOGY A neuron is like a person at a busy tea stall who listens to many friends at once, decides whether the news is important enough, and then shouts it to the next group. The more often a particular friend turns out to be right, the more you trust that friend — that is how connections get stronger.

SAY IT LIKE THIS “A neuron collects signals, makes one decision, and passes it on. When we practise something, the connections used get stronger, and that is a key part of how the brain learns. Computers copy this using numbers called weights.”

SLIDE 3 Nature Inspired the Machine

This slide is a translation table, part by part. Each piece of a biological neuron has a maths twin:

- Dendrites → **inputs** (x) · Synaptic strength → **weights** (w) · Soma → **summation** (Σ) · Firing threshold → **activation** (f) · Axon → **output** (y).
- **1943, McCulloch & Pitts** proposed the first mathematical model of a neuron: a simple on/off threshold unit. This is the “birth” of the artificial neuron.

ANALOGY A judge listening to advisers. Each adviser gives an opinion (input), the judge trusts some advisers more than others (weights), adds up the opinions (summation), and only gives a verdict if the total is strong enough (activation).

SAY IT LIKE THIS “Every part of the brain neuron has a matching part in our model — dendrites become inputs, synaptic strength becomes weights, the soma becomes a sum, the firing threshold becomes the activation, and the axon becomes the output.”

SLIDE 4 The Artificial Neuron

This is the most important slide of Part 1. An artificial neuron does just two steps. **Step 1**: multiply each input by its weight, add them all up, and add the bias. This total is called **net**. **Step 2**: pass net through an activation function f to get the output y .

$$\text{net} = \sum w_i \cdot x_i + b \quad | \quad y = f(\text{net})$$

The live demo on the slide: $x = [1.0, 0.0, 1.0, 0.5]$, $w = [0.5, 0.75, -0.25, 0.25]$, $b = 0.25$. So $\text{net} = 0.5 + 0 - 0.25 + 0.125 + 0.25 = \mathbf{0.625} \approx \mathbf{0.63}$. The sigmoid then squashes 0.63 into **0.65**. Click a weight on the slide and watch y change — that is the whole idea of learning in one picture. Thick lines = big weights; violet = negative weights.

Activation	What it does	Think of it as
Step	Output is exactly +1 or -1 (or 0/1)	A light switch: on or off
Sigmoid	Smoothly squeezes any number into 0 to 1	A dimmer knob
Tanh	Same idea, but squeezes into -1 to 1	A dimmer that can also go “negative”
ReLU	$\max(0, x)$: negatives become 0, positives pass	A one-way gate that blocks negatives

ANALOGY Deciding whether to go to a party. Inputs: “Is my best friend going?” (1 = yes), “Do I have an exam tomorrow?” (1 = yes). Weights: how much each matters to you (friend: +0.8, exam: -1.5). Bias: your general mood (a social person starts with a positive bias). You add it all up; if the total is high enough, you go.

SAY IT LIKE THIS “A neuron just multiplies, adds, and decides. The weights say how important each input is, the bias shifts how easily it fires, and the activation function turns the total into the final answer.”

SLIDE 5 One Neuron Is Not Enough (Architecture)

A single neuron can only make simple decisions. If we connect many neurons in **layers**, we get a neural network. Information flows forward, layer by layer. Three kinds of layer: **input** (takes in the data), **hidden** (finds useful patterns inside), **output** (gives the answer). Every line between neurons has a weight w_{ij} , and what the network “knows” is stored in these numbers (plus the biases).

ANALOGY A company: the receptionists (input layer) collect the raw information, the analysts (hidden layers) find the patterns in it, and the manager (output layer) announces the decision. Nobody holds the knowledge alone — it is spread across all the relationships between them.

SAY IT LIKE THIS “One neuron is too simple, so we stack many in layers. Data enters at the input, the hidden layers find patterns, and the output gives the answer. All the knowledge sits in the weights.”

Part 2 · Learning

Slides 6–13: how a network changes its weights until it is right

SLIDE 6 Learning Rules

A network starts with random weights, so its answers are wrong. “Learning” simply means changing the weights. A **learning rule** is the recipe that says *how much* to change each weight (Δw). New weight = old weight + Δw . Symbols: η (eta) = learning rate, a step size; **t** = target (right answer); **y** = what the network said; **x** = input.

Rule	Formula	In simple words
Hebbian	$\Delta w = \eta \cdot x \cdot y$	“Neurons that fire together wire together.” Two friends who are always together become closer. No answer key needed.
Perceptron	$\Delta w = \eta \cdot (t - y) \cdot x$	Change weights only when the answer is wrong, and in the direction that fixes it.
Delta	$\Delta w = \eta \cdot (t - y) \cdot f'(\text{net}) \cdot x$	Like perceptron, but the size of the fix depends on how big the error is. (Widrow-Hoff / LMS is the case where $f' = 1$.)
Competitive	$\Delta w = \eta \cdot (x - w)$, winner only	Only the best-matching neuron learns, by moving its weights towards the input. Like only the top student getting extra coaching.

Do not mix up three words: **rule** = how one weight is changed; **paradigm** = what kind of feedback you have (slide 7); **algorithm** = the full step-by-step training procedure (slide 13).

ANALOGY A chef tasting soup. The soup is “the output”, the recipe book is the “rule” that says how much salt to add or remove, and the pinch you actually add is Δw . You taste, adjust, taste again, until it is right.

SAY IT LIKE THIS “Learning means changing weights, and the rule is the recipe for how big each change should be.”

SLIDE 7 Three Ways a Machine Can Learn

The kind of feedback decides the learning type.

Type	Feedback	Example	Algorithms
Supervised	Told exactly how wrong it was	Recognising handwritten digits	Perceptron, delta rule, backpropagation
Unsupervised	No answers, only the data’s own pattern	Grouping similar things	Hebbian, competitive learning (Kohonen SOM)
Reinforcement	A reward or penalty, often later	A program learning to play a game	Weights updated from a reward (critic) signal

ANALOGY Learning to cook. **Supervised:** a teacher tastes every dish and tells you exactly what is missing. **Unsupervised:** nobody tells you anything; you taste lots of dishes and notice which ones are similar. **Reinforcement:** you cook, and the guests only say “great!” or “yuck!” afterwards.

SAY IT LIKE THIS “Supervised has an answer key, unsupervised has no answers and finds patterns by itself, and reinforcement learns from rewards and penalties.”

SLIDE 8

The Perceptron

Rosenblatt, 1958 — the first neuron that can learn. It is the artificial neuron from slide 4 with a **step function**: weighted sum, then output +1 if the sum is ≥ 0 , otherwise -1. Geometrically it draws a straight line (**decision boundary**, $w \cdot x + b = 0$) that separates class +1 from class -1.

ANALOGY A bouncer at a club door with a checklist. He adds up points for each rule (age, ticket, dress), and if the total is at least zero you are in (+1), else out (-1). The limitation: he can only draw *one straight line* between “in” and “out”. If the red and blue marbles on the table are mixed so that no single straight stick can separate them (the classic example is XOR), he fails.

SAY IT LIKE THIS “The perceptron makes a yes-or-no decision using one straight line. It can learn only problems a straight line can solve, which is exactly why we need multilayer networks later.”

SLIDE 9

A Tiny Learning Step (Worked Example)

One example, one correction, with numbers you can check. Given: $x = [1, 0, 1]$, $w = [0.2, -0.3, 0.1]$, $b = 0.1$, target $t = -1$, learning rate $\eta = 0.1$.

Step	Calculation	Result
1. Net input	$0.2 \cdot 1 + (-0.3) \cdot 0 + 0.1 \cdot 1 + 0.1$	net = 0.4
2. Output	net ≥ 0 , so fire	$y = +1$
3. Error	$e = t - y = -1 - (+1)$	$e = -2$ (wrong!)
4. Update weights	$\Delta w_i = \eta \cdot e \cdot x_i = 0.1 \cdot (-2) \cdot [1, 0, 1]$	$\Delta w = [-0.2, 0, -0.2]$
5. New values	$w = [0.2 - 0.2, -0.3, 0.1 - 0.2]$; $b = 0.1 - 0.2$	$w = [0, -0.3, -0.1]$, $b = -0.1$
6. Check	net = $0 \cdot 1 + (-0.3) \cdot 0 + (-0.1) \cdot 1 + (-0.1)$	net = -0.2, so $y = -1$ ✓

Repeat this over all the examples and the weights settle on a correct solution — as long as a straight line can separate the classes.

ANALOGY A dart player. The dart lands too high (error). He does not throw randomly; he adjusts his aim slightly in the opposite direction, and throws again. Notice that input x_2 was 0 — a silent input is not blamed, so its weight does not change.

SAY IT LIKE THIS “The network guessed +1 but the truth was -1, so it was wrong. We correct each weight by learning rate times error times input. After one correction, the same input gives -1, which is right.”

SLIDE 10 From “Wrong” to “How Wrong?” (Delta Rule)

The perceptron only knows *right or wrong*, so its error can only be -2 , 0 or $+2$. The delta rule uses a smooth output, so the error can be **any size**, and a bigger mistake gives a bigger correction. It works by **gradient descent**: slide downhill on the error surface $E = \frac{1}{2}(t - y)^2$ until you reach the bottom (smallest error).

$$w_i(\text{new}) = w_i + \eta \cdot (t - y) \cdot f'(\text{net}) \cdot x_i$$

$f'(\text{net})$ is the **slope** of the activation function — it tells how sensitive the output is at that point. The square in $E = \frac{1}{2}(t - y)^2$ makes big mistakes count much more than small ones.

ANALOGY Pass/fail versus marks out of 100. A teacher who only says “fail” gives you little to work with; a teacher who says “you got 32, you were far off, especially in section B” helps you fix things in proportion. Gradient descent is like walking down a foggy hill: you cannot see the valley, but you feel the slope under your feet and keep stepping downhill.

SAY IT LIKE THIS “The perceptron says right or wrong; the delta rule says how wrong and in which direction, so corrections are proportional. The learning rate η is the step size: too small is slow, too large overshoots and becomes unstable.”

SLIDE 11 MLP — Scaling the Perceptron

A **multilayer perceptron** stacks neurons in layers, with **hidden layers** in the middle. Early hidden neurons find simple features and later ones combine them. A hidden layer with a non-linear activation can solve problems no straight line can (for example XOR). Do not confuse the two names: **MLP = the architecture** (what the network is), **backpropagation = the training algorithm** (how it learns).

ANALOGY Recognising a handwritten “8”. Layer 1 spots tiny strokes and edges, layer 2 combines them into loops and curves, and the output layer says “that is an 8”. Also: one stick can only make a straight fence, but many short sticks joined together can approximate any curve. If the activation were linear, all the layers would collapse into one — that is why non-linear activation matters.

SAY IT LIKE THIS “An MLP is just perceptron-like neurons stacked in layers. The hidden layers build up from simple to complex features, which lets it solve problems that a single perceptron cannot.”

SLIDE 12 How Does a Multilayer Network Learn? (Backpropagation, idea)

Backpropagation works in a loop of three phases, which you can watch in the animation: **Forward** — data flows forward and the network makes a prediction $\hat{y}$. **Backward** — the error is sent back, to “blame” each weight for its share of the mistake. **Update** — each weight is nudged to reduce the error.

$$w_{ij} \leftarrow w_{ij} - \eta \cdot \partial E / \partial w_{ij}$$

Reading the formula: $\partial E / \partial w$ means “how much does the error change if I nudge this one weight a tiny bit?” The minus sign means we move the weight in the direction that makes the error smaller.

ANALOGY A group project that got a poor grade. The teacher tells the group only the final grade (the error). Then you trace back: who wrote which part, and how much did each person contribute to the mistake? Each person then fixes their part in proportion to their share of the blame. The hidden-layer neurons are the team members who never see the final grade directly — the blame must travel back to them.

SAY IT LIKE THIS “Backpropagation first runs the data forward, measures the error at the output, then sends the blame backwards through the layers so every weight knows how to change.”

SLIDE 13 **Backpropagation — Step by Step**

Here is the full **algorithm** in 8 steps. The new idea is **δ (delta) = “blame”** for each neuron.

#	Step	Meaning
1	Initialise	Small random weights and biases
2	Forward pass	Push inputs through each layer to get the output
3	Compute error	$E = \frac{1}{2} \sum (t - y)^2$
4	Output δ	$\delta_k = (t_k - y_k) \cdot f'(\text{net}_k)$ — blame at the output layer
5	Hidden δ	$\delta_j = f'(\text{net}_j) \cdot \sum \delta_k w_{jk}$ — a hidden neuron’s blame = the blames of the neurons it feeds, weighted by its connections to them
6	Update weights	$\Delta w_{ij} = \eta \cdot \delta_j \cdot x_i$ (same as $-\eta \partial E / \partial w_{ij}$)
7	Repeat	Go through all examples — one full pass is one epoch
8	Stop	When the error is small enough (or a maximum number of epochs is reached)

ANALOGY Think of a restaurant that gets a bad review: “soup is too salty”. The owner blames the head chef (output δ). The head chef then blames the sous-chefs in proportion to how much each one’s work affected the soup (hidden δ). The more a neuron influenced the final result, the more blame it receives.

SAY IT LIKE THIS “We start with small random weights, push data forward, compute the error, work out the blame at the output and then backwards in each hidden layer, update every weight, and repeat until the error is small enough.”

Part 3 · Memory and Applications

Slides 14–18: a network that remembers, where networks are used, and the summary

SLIDE 14 **When a Network Becomes a Memory (Hopfield)**

John Hopfield, 1982. This is a different kind of network: it is for **storing and recalling patterns**, not for learning from examples with answers. Every neuron is connected to every other neuron (but not to itself), and each link works equally in both directions ($w_{ij} = w_{ji}$). Patterns are stored in the weights using Hebb’s rule, and a network of N neurons can reliably store about $0.14 \cdot N$ patterns. Neurons update one at a time, and the network’s **energy** keeps dropping until it settles into a stable state.

ANALOGY A bumpy landscape with valleys. Each stored pattern is the bottom of a valley. If you drop a ball (a messy, noisy pattern) anywhere nearby, it rolls down into the nearest valley and stops there — that valley is the remembered pattern. “Energy” is simply the height of the ball: the network always moves downhill. A few false valleys (spurious states) can also exist.

SAY IT LIKE THIS “A Hopfield network stores patterns as memories. If you show it a damaged version, it settles back to the closest stored one, because it keeps lowering its energy until it cannot go lower.”

SLIDE 15

Remembering From Incomplete Information (Associative Memory)

An associative memory recalls by **content**, not by address: a partial clue is enough. On the slide you can watch the violet “mistakes” in a noisy pattern get fixed one cell at a time by the Hopfield dynamics.

Type	What it does	Everyday example
Auto-associative	A damaged pattern is repaired into the original	Fixing a smudged photo, autocomplete of a half-typed word
Hetero-associative	One pattern recalls a different, paired pattern (key → value). Example network: BAM	Name → phone number

ANALOGY Your brain: hear the first two seconds of a song and you can finish the whole song. Computers normally need an exact file name (address) — like a library where you must know the shelf number. An associative memory is like a librarian who finds the book when you say, “the one with a red cover about space.”

SAY IT LIKE THIS “The network recalls from content: give it a partial or noisy pattern and it completes it. Auto-associative repairs the same pattern, hetero-associative links one pattern to a different one.”

SLIDE 16

Pattern Recognition

The network turns a raw pattern into **features** (a feature vector), then decides which **class** it belongs to. On the slide the network sees a handwritten letter and gives probabilities: A 94%, R 4%, H 2% (the numbers are illustrative). Real uses: reading handwriting (OCR), recognising who is speaking, fingerprint and face identification.

ANALOGY How you spot a friend in a crowd from far away. You do not compare every pixel; you notice a few features (height, walk, hair) and your brain says “that is Rahul”. The network does the same: raw input → features → decision.

SAY IT LIKE THIS “Pattern recognition is the most common use of neural networks: it converts raw data into features and chooses the most likely class.”

SLIDE 17

Forecasting and Image Processing

Forecasting: the network learns from historical data how values change over time, and predicts future values (for example the line on the slide continues beyond “NOW”). **Image processing**: pixels are converted to features and the network gives a result, such as the class “Z” with about 97% (illustrative). Typical tasks: removing noise, outlining objects, medical scans.

ANALOGY Forecasting is like an experienced cricket commentator who has watched thousands of innings: given the first 20 overs, he projects the final score from the pattern he has seen before. Image processing is like a doctor reading an X-ray: the image becomes a few meaningful clues, and the clues become a diagnosis.

SAY IT LIKE THIS “For forecasting, the network learns how a series behaves over time and extends it. For images, it turns pixels into features and then into a decision, which is useful for cleaning, outlining and medical scans.”

SLIDE 18

What We Learned (Takeaways)

The six points are a summary of the whole unit. Use them to close:

- A neuron is just arithmetic: $y = f(\sum w_i x_i + b)$.
- Learning means changing weights — experience gets stored as numbers.
- The kind of feedback decides the learning type: supervised, unsupervised or reinforcement.
- The perceptron learns from mistakes; the delta rule makes the fix bigger when the mistake is bigger.
- Backpropagation sends the error backward, so multilayer networks can learn.
- A network can act as a memory — Hopfield networks recall whole patterns from partial clues.

ANALOGY It is the chapter summary you give a friend at the exit of the exam hall: neuron → learning → feedback types → perceptron/delta → backprop → memory.

SAY IT LIKE THIS “From biological inspiration to computational intelligence — thank you, we are happy to take your questions.”

Quick Revision

Formulas, key words, and questions your professor may ask

Formula cheat sheet

Idea	Formula	Slide
Neuron	$\text{net} = \sum w_i x_i + b ; y = f(\text{net})$	4
Weight update (any rule)	$w(\text{new}) = w(\text{old}) + \Delta w$	6
Hebbian	$\Delta w = \eta \cdot x \cdot y$	6
Perceptron	$\Delta w_i = \eta \cdot (t - y) \cdot x_i$	6, 9
Delta	$\Delta w_i = \eta \cdot (t - y) \cdot f'(\text{net}) \cdot x_i ; E = \frac{1}{2}(t - y)^2$	10
Competitive	$\Delta w = \eta \cdot (x - w)$, winner only	6
Backprop update	$w_{ij} \leftarrow w_{ij} - \eta \cdot \partial E / \partial w_{ij}$	12
Output / hidden δ	$\delta_k = (t_k - y_k) f'(\text{net}_k)$; $\delta_j = f'(\text{net}_j) \sum \delta_k w_{jk}$	13
Hopfield	$w_{ij} = w_{ji}$, $w_{ii} = 0$; capacity $\approx 0.14 \cdot N$	14

Glossary in one line each

Term	Meaning
Neuron	A unit that adds weighted inputs and decides an output
Weight / bias	Importance of an input / a built-in nudge
Activation function	The decision step: step, sigmoid, tanh, ReLU
Learning rate (η)	Step size of each correction
Epoch	One full pass through all training examples
Hidden layer	A layer between input and output that finds features
Decision boundary	The line that separates two classes
Gradient descent	Repeatedly stepping downhill on the error
δ (delta)	The “blame” assigned to a neuron in backpropagation
Energy (Hopfield)	A number that always goes down as the network settles

Questions you may be asked — with short answers

Q. Why can't a single perceptron solve XOR?

A. Because XOR cannot be separated by one straight line, and a perceptron can only draw one. A hidden layer with non-linear activation fixes this.

Q. What is the difference between the perceptron rule and the delta rule?

A. The perceptron only cares whether the answer is right or wrong (error -2 , 0 , $+2$); the delta rule uses the size of the error and the slope of the activation, so corrections are proportional.

Q. Why do we need an activation function?

A. It adds the decision/non-linearity. Without it, many layers would collapse into one linear layer, so the network could not learn curved patterns.

Q. What does backpropagation actually do?

A. It computes how much each weight contributed to the final error (using the chain rule, layer by layer from the output backwards) and then changes each weight to reduce it.

Q. What is the role of the learning rate?

A. It sets the step size. Too small makes learning very slow; too large overshoots the minimum and becomes unstable.

Q. Is a Hopfield network trained with backpropagation?

A. No. Its weights are set directly from the patterns using Hebb's rule, and recall happens by letting the network settle to lower energy.

Q. Which paradigm does each algorithm belong to?

A. Perceptron, delta rule and backpropagation are supervised; Hebbian and competitive learning are unsupervised; reward-based updates are reinforcement learning.

Q. Why does the Hopfield network sometimes recall a wrong pattern?

A. Because besides the stored patterns there can be false stable states (spurious states), and if too many patterns are stored (more than about $0.14 \cdot N$) recall becomes unreliable.

Presenting tips. Point at the slide while you speak. Use the analogy first, the formula second. If you forget a number, say what it means instead ("the weights are changed in proportion to the error"). Practise the live demos once: click the weights on slide 4, and watch the backpropagation and Hopfield animations before you present.